BCSE205L

Computer Architecture and Organization

Module 3 – Instruction Sets and Control Unit

Module:3	Instruction Sets and Control Unit	9 Hours
Computer Instructions: Instruction sets, Instruction Set Architecture, Instruction formats, Instruction set categories - Addressing modes - Phases of instruction cycle – ALU - Data-path and control unit: Hardwired control unit and Micro programmed control unit - Performance metrics: Execution time calculation, MIPS, MFLOPS.		

Dr. P.Keerthika

Associate Professor

School of Computer Science and Engineering

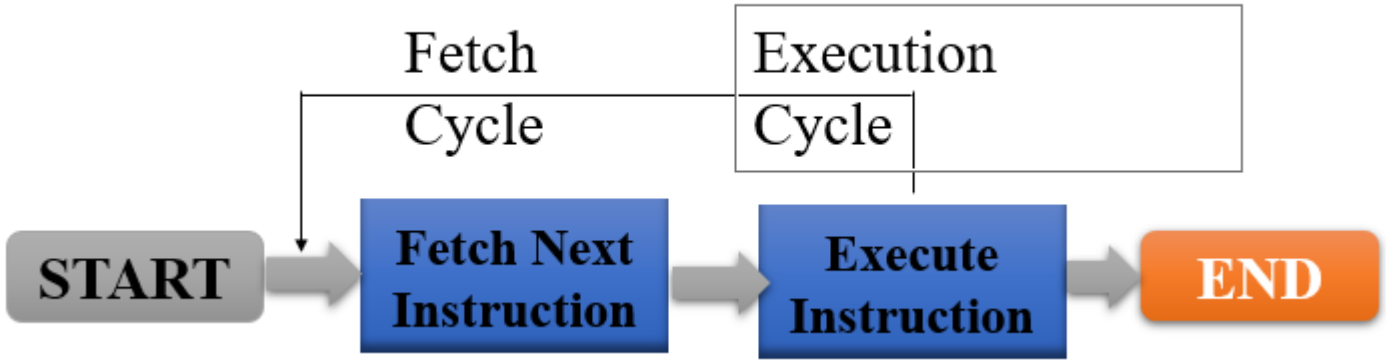
Vellore Institute of Technology, Vellore

Computer Instructions: Instruction sets

- **Instruction**
  - Is a statement by which the operation of CPU is determined.
  - These instructions referred as “Machine instructions or computer Instructions”
- **Elements of Machine instruction**
  - Operation code
  - Source operand reference
  - Result operand Reference
  - Next instruction reference

Computer Instructions: Instruction sets

- **Program**
  - is sequence of instructions which operates on data to perform certain tasks.
- **Instruction**
  - Is a statement by which the operation of CPU is determined
- **Instruction Cycle**



Instruction Set Architecture

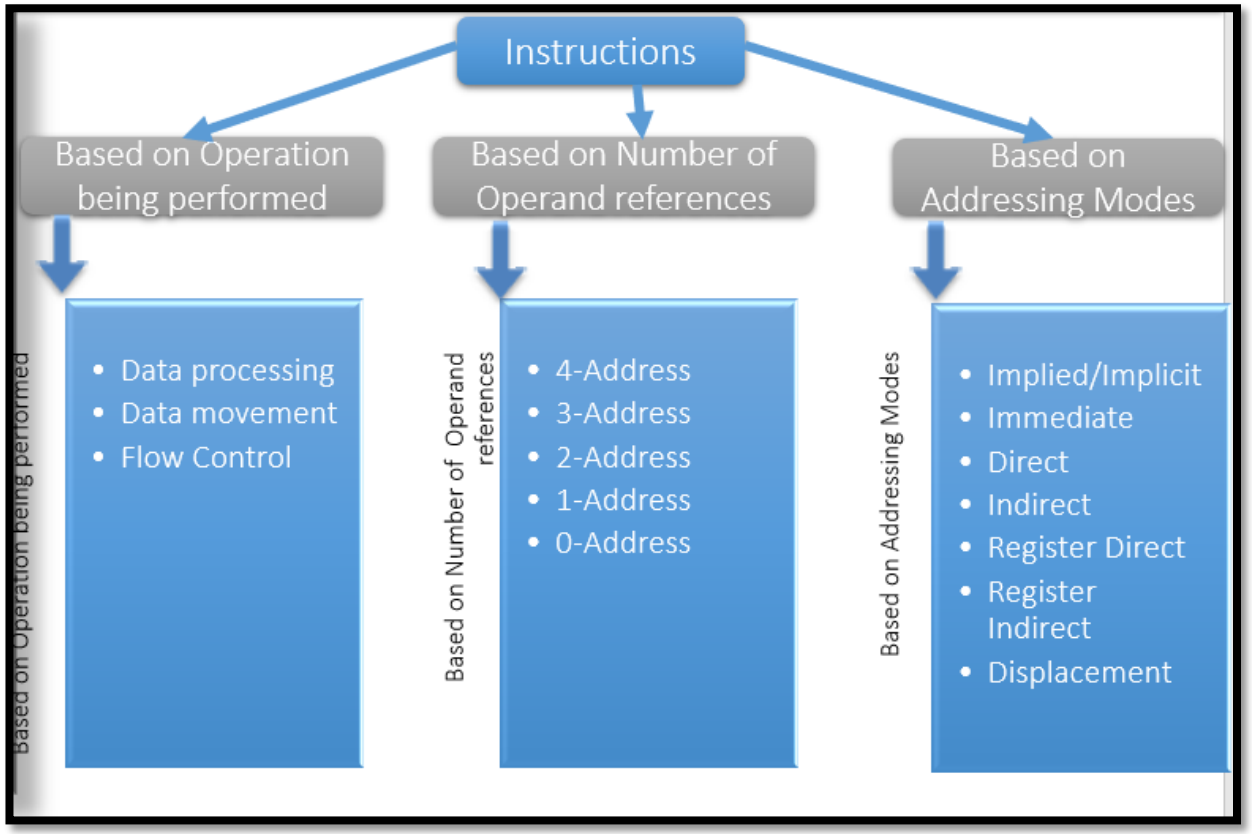
- Instruction Set Architecture (ISA) - defines how the CPU is controlled by the software
- ISA acts as an interface between the hardware and the software, specifying both what the processor is capable of doing as well as how it gets done
- It defines the supported data types, the registers, how the hardware manages main memory, key features (such as virtual memory), which instructions a microprocessor can execute, and the input/output model of multiple ISA implementations

Instruction formats

- Each instruction is represented by sequence of bits
- The instruction is divided into two fields
  - Opcode field
  - Operand field
- This operand field further divided into one to four fields.
- This layout of the instruction is known as the “Instruction Format”

Instruction Set category

- Instruction Set is categorized into types based on
  - Operation performed
  - Number of operand addresses
  - Addressing modes.



Instruction Set category

- **Based on Operation being performed**
- Data movement → Move data from a memory location or register to another memory location or register without changing its form.
  - Memory - LOAD, STORE, MOV
  - I\O Instructions - IN, OUT
- Data processing→ Arithmetic and logic (ALU) instructions - Changes the form of one or more operands to produce a result stored in another location
  - Arithmetic - Add, Sub, MUL
  - logic Instructions - AND, OR,

Instruction Set category

- **Based on Operation being performed**
- Flow Control→ Any instruction that alters the normal flow of control from executing the next instruction in sequence
  - Conditional
    - JNZ, JZ
  - Un Conditional
    - Jump

Instruction Set category

- **Based on Number of operand addresses**
- Instruction Set categorized into four categories based on number of operand address in the instruction.
  - 4-Address Instruction
  - 3-Address Instruction
  - 2-Address Instruction
  - 1-Address Instruction
  - 0-Address Instruction

Example: add M1,M2,M3, nexti

$M(1) \leftarrow M(2)+M(3)$

Instruction Set category

$$X = (A + B) * (C + D)$$

Three Address Instruction

Example:

- $X = (A + B) * (C + D)$
- ADD R1, A, B $R1 \leftarrow M[A] + M[B]$
  ADD R2, C, D $R2 \leftarrow M[C] + M[D]$
  MUL X, R1, R2 $M[X] \leftarrow R1 * R2$

Two Address Instruction

- $X = (A + B) * (C + D)$
- MOV R1, A
  ADD R1, B
  MOV R2, C
  ADD R2, D
  MUL R1, R2
  MOV X, R1

Instruction Set category

One Address Instruction

- X = (A + B) * (C + D)

LOAD A	AC <- M[A]
ADD B	AC <- AC+ M[B]
STORE T	M[T]<- AC
LOAD C	AC <- M [C]
ADD D	AC<- AC+ M[D]
MUL T	AC <- AC * M[T]
STORE X	M[X]<- AC

- X = (A + B) * (C + D)

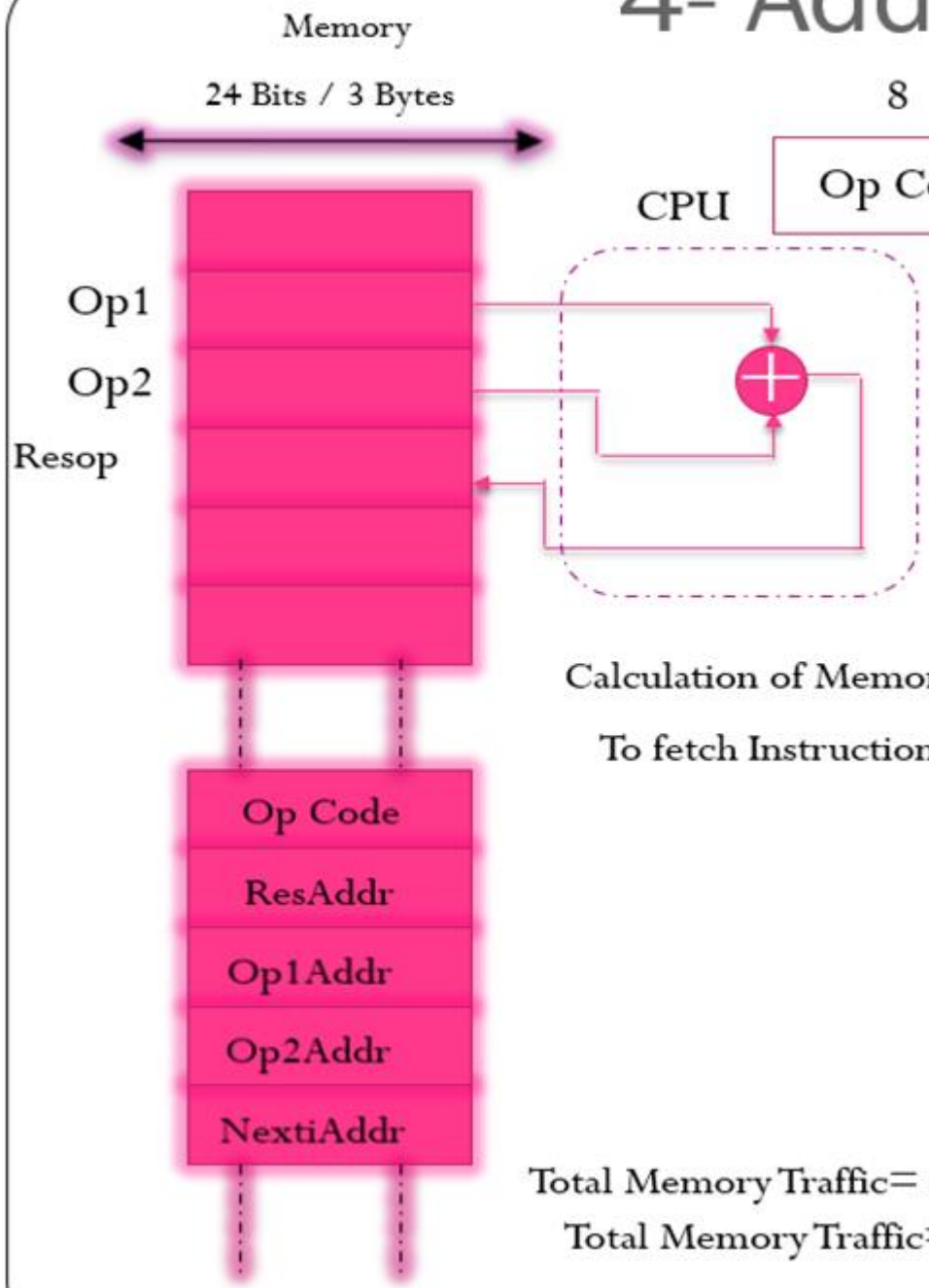
PUSH A	TOS<- A
PUSH B	TOS <-B
ADD	TOS<- A+B
PUSH C	TOS<- C
PUSH D	TOS<-D
ADD	TOS<-C+D
MUL	TOS<-(C+D)* (A+B)
POP X	M[X]<- TOS

Zero Address Instruction (Don’t use address field for instruction

Calculation of Memory traffic

- **Assumptions**
  - 24-bit memory address (3 bytes)
  - 128 instructions (7 bits rounded to 1 byte)

4- Address Instruction



8	24	24	24	24
Op Code	ResAddr	Op1Addr	Op2Addr	NextiAddr

Example: add M1,M2,M3, nexti

M(1)←M(2)+M(3)

Memory Required to store an Instruction: 5 x 3 bytes = 15 Bytes

Memory Required to Encode an Instruction:

1Byte+ 4 x 3 bytes = 13 Bytes

Calculation of Memory Accesses

To fetch Instruction itself

Opcode=1
Op1Addr=1
Op2Addr=1
ResAddr=1
NextiAddr=1
Total=5

To Fetch Operands and Execute the instruction

Op1=1
Op2=1
Res=1
Total=3

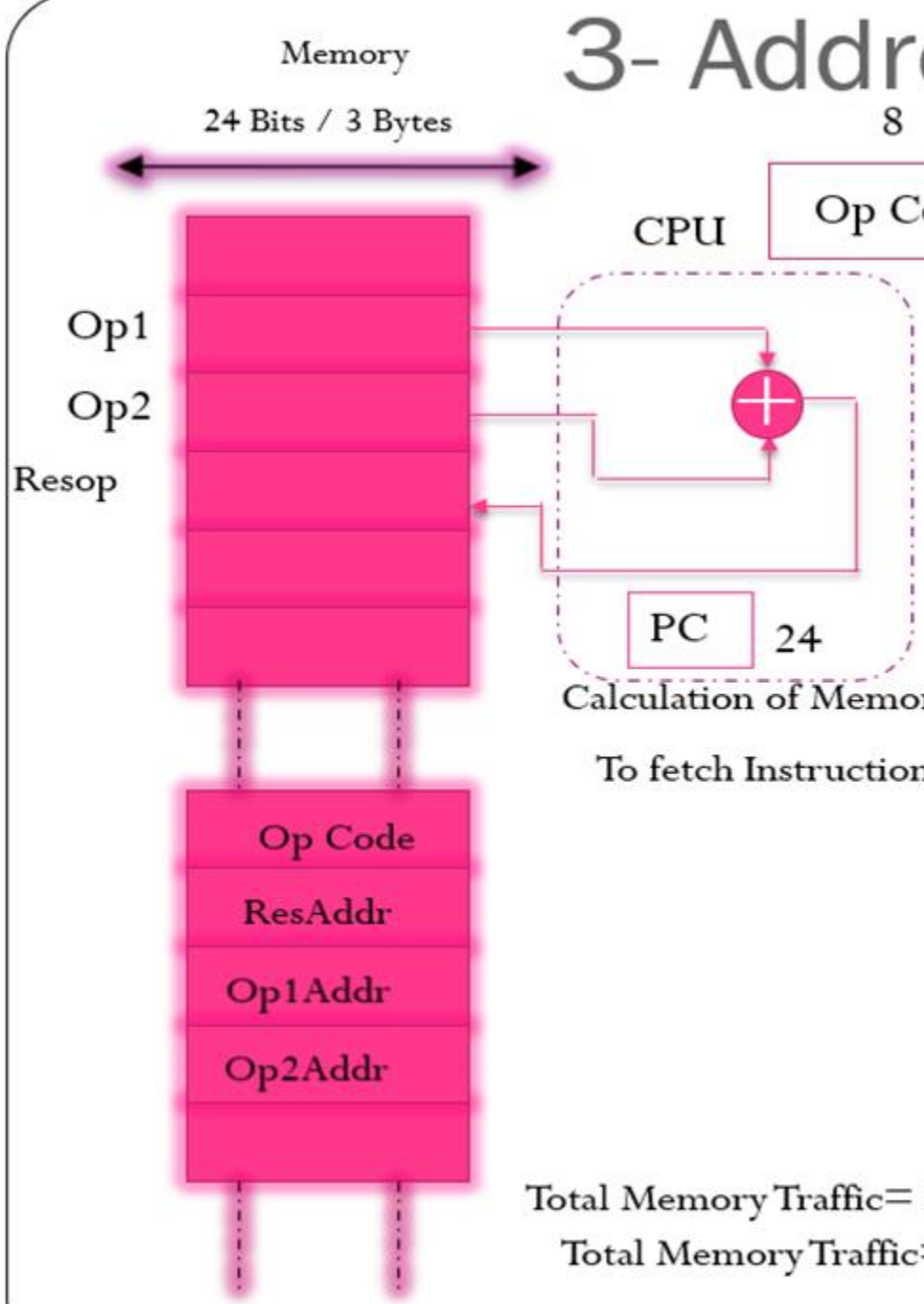
Total Memory Traffic= No. of M/A to fetch + No. of M/A to Execute

Total Memory Traffic= 5 + 3 =8

4 – address Instruction Design

- Because of the large instruction word size and number of memory accesses ,the 4- address machine and instruction format is not seen in the machine design.
- Although the 4-address structure is used internally in some implementations of computer control units. This kind of controller implementations is known as *microcoded Control*.

3- Address Instruction



8	24	24	24
Op Code	ResAddr	Op1Addr	Op2Addr

Example: add M1,M2,M3

$M(1) \leftarrow M(2)+M(3)$

Memory Required to store an Instruction: 4 x 3 bytes = 12 Bytes

Memory Required to Encode an Instruction:

1Byte+ 3 x 3 bytes = 10 Bytes

Calculation of Memory Accesses

To fetch Instruction itself

Opcode=1
Op1Addr=1
Op2Addr=1
ResAddr=1
Total=4

To Fetch the Operands and Execute the Instruction

Op1=1
Op2=1
Res=1
Total=3

Total Memory Traffic= No. of M/A to fetch + No. of M/A to Execute

Total Memory Traffic= 4+ 3 =7

3 – address Instruction Design

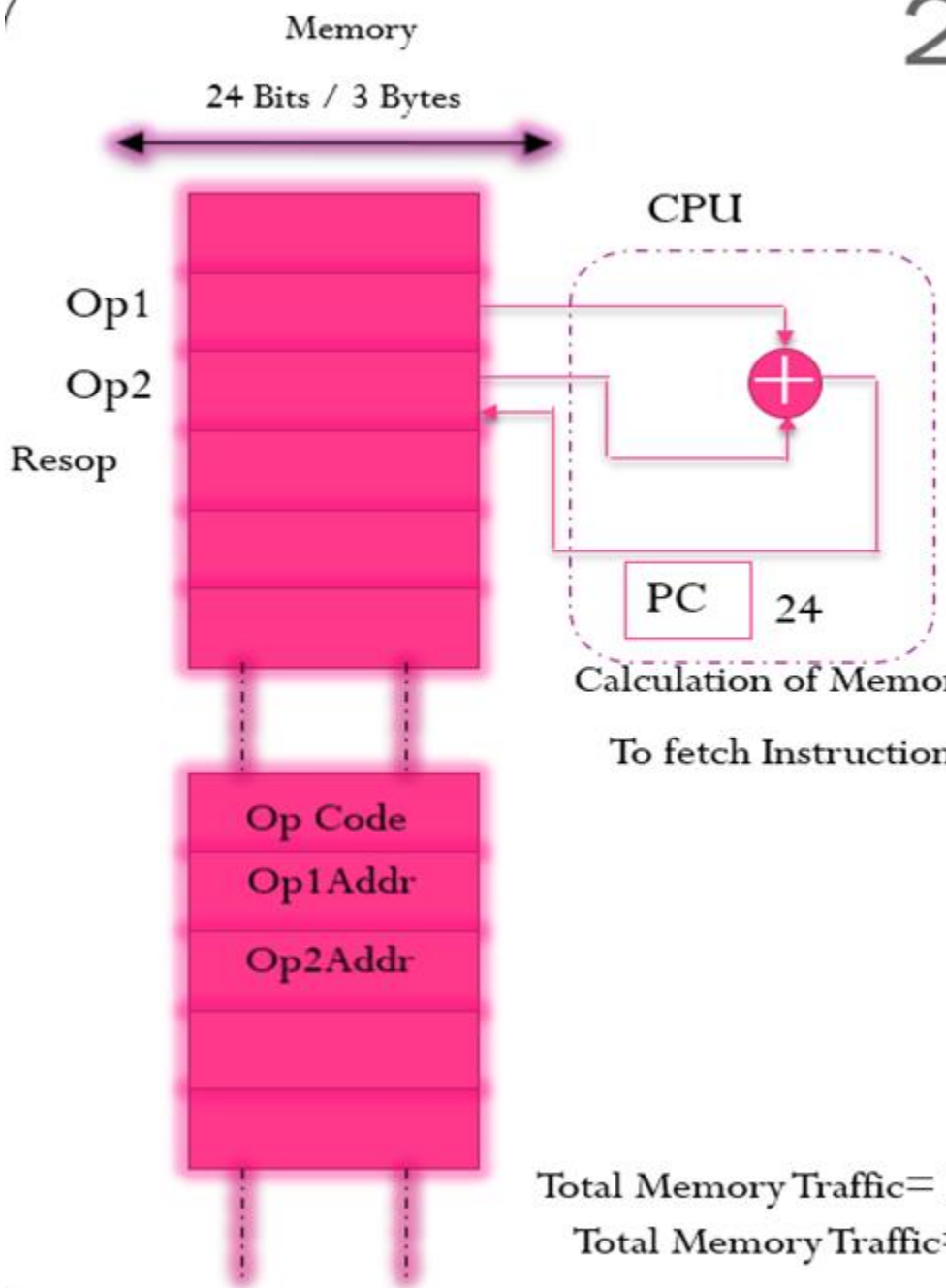
3-Address instruction:

- Address of next instruction kept in processor state register—the PC (Except for explicit Branches/Jumps)
- Rest of addresses in instruction
- This Instruction will require 3X3+1= 10 bytes to encode a 3-address ALU instruction.

The number of *memory access* are required for a 3-address instruction:

- Four words will be transferred to the CPU when the instruction itself is fetched.= 4
- Then the two words representing the operands themselves need to be fetched into the CPU = 2
- And after the addition has been performed, the result needs to be written back to memory = 1 **Total =07**

2- Address Instruction



8	24	24
Op Code	Op1Addr	Op2Addr

Example: add M2,M3

$M(2) \leftarrow M(2)+M(3)$

Memory Required to store an Instruction:

3 x 3 bytes = 09 Bytes

Memory Required to Encode an Instruction:

1Byte+ 2 x 3 bytes = 7 Bytes

Calculation of Memory Accesses

To fetch Instruction itself

Opcode=1
Op1Addr=1
Op2Addr=1

Total=3

To Fetch the Operands and Execute the Instructions

Op1=1
Op2=1
Res=1

Total=3

Total Memory Traffic= No. of M/A to fetch + No. of M/A to Execute

Total Memory Traffic= 3+ 3 =6

2 – address Instruction Design

2-address Instruction :

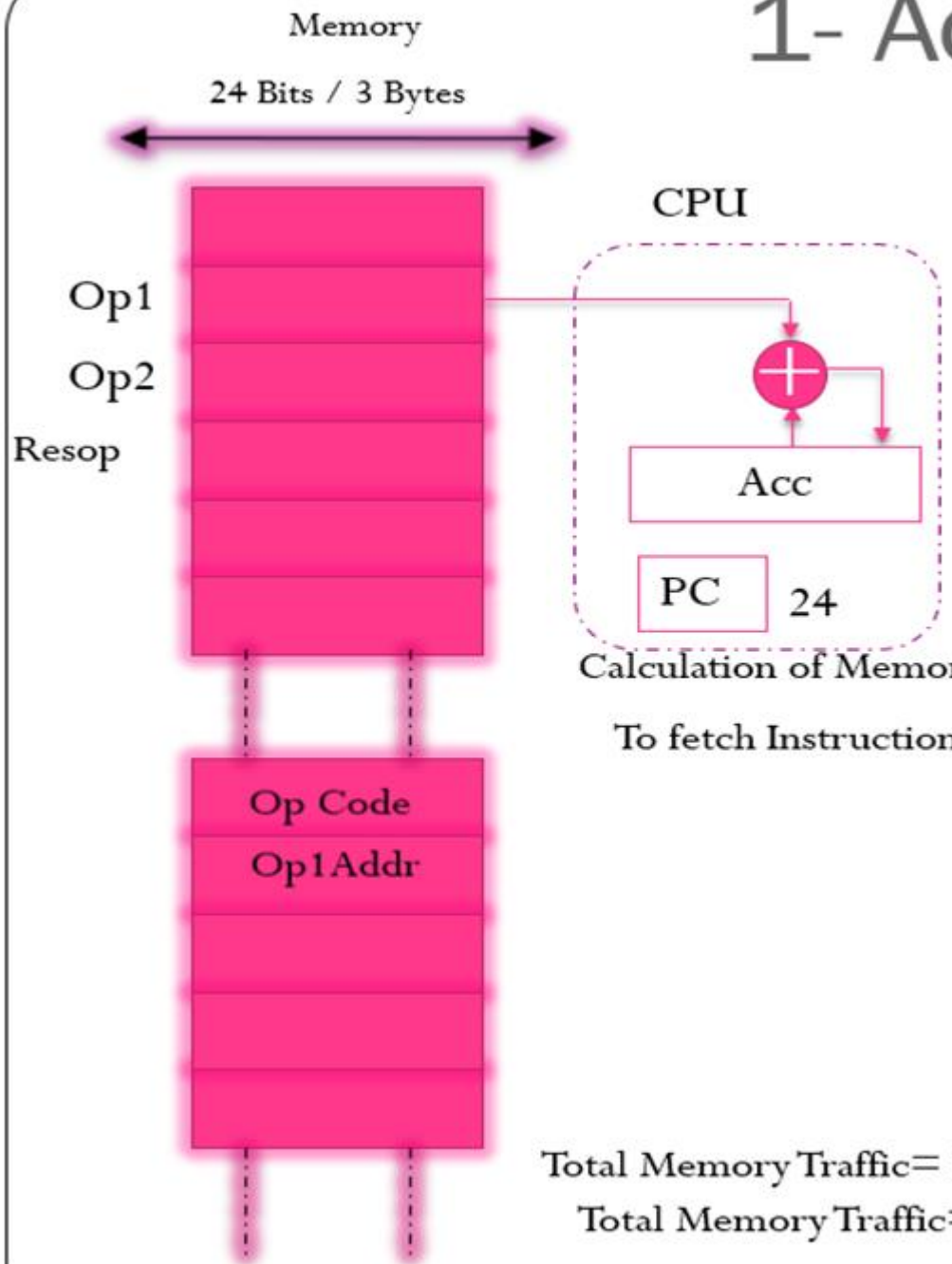
- Result overwrites Operand 2
- Needs only 2 addresses in instruction but less choice in placing data
- This Instruction will require 2X3+1= 7 bytes to encode a 2-address ALU instruction.

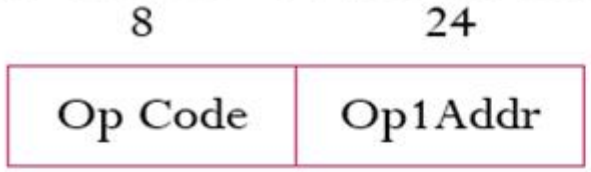
The number of *memory access* are required for a 2-address instruction:

- Three words will be transferred to the CPU when the instruction itself

  is fetched. = 3
- Then the two words representing the operands themselves need to be fetched into he CPU and after the addition has been performed, Result overwrites Operand =3
- **Total= 06**
- **add Op1Addr Op2Addr**

1- Address Instruction





Example: add M2

Memory Required to store an Instruction: 2 x 3 bytes = 06 Bytes

Memory Required to Encode an Instruction:

1Byte+ 1 x 3 bytes = 4Bytes

Calculation of Memory Accesses

To fetch Instruction itself

Opcode=1

Op1Addr=1

Total=2

To Fetch Operands adn Execute Instruction

Op1=1

Total=1

Result stored in Accumulator – Hence No Memory access required

Total Memory Traffic= No. of M/A to fetch + No. of M/A to Execute

Total Memory Traffic= 2+ 1 =3

1 – address Instruction Design

1-address Instruction :

- Special CPU register, the accumulator, supplies 1 operand and stores result
- One memory address used for other operand
- Need instructions to load and store operands:
  - LDA OpAddr
  - STA OpAddr

- This Instruction will require 1X3+1= 4 bytes to encode a 1-address ALU instruction

The number of *memory access* are required for a 1-address instruction:

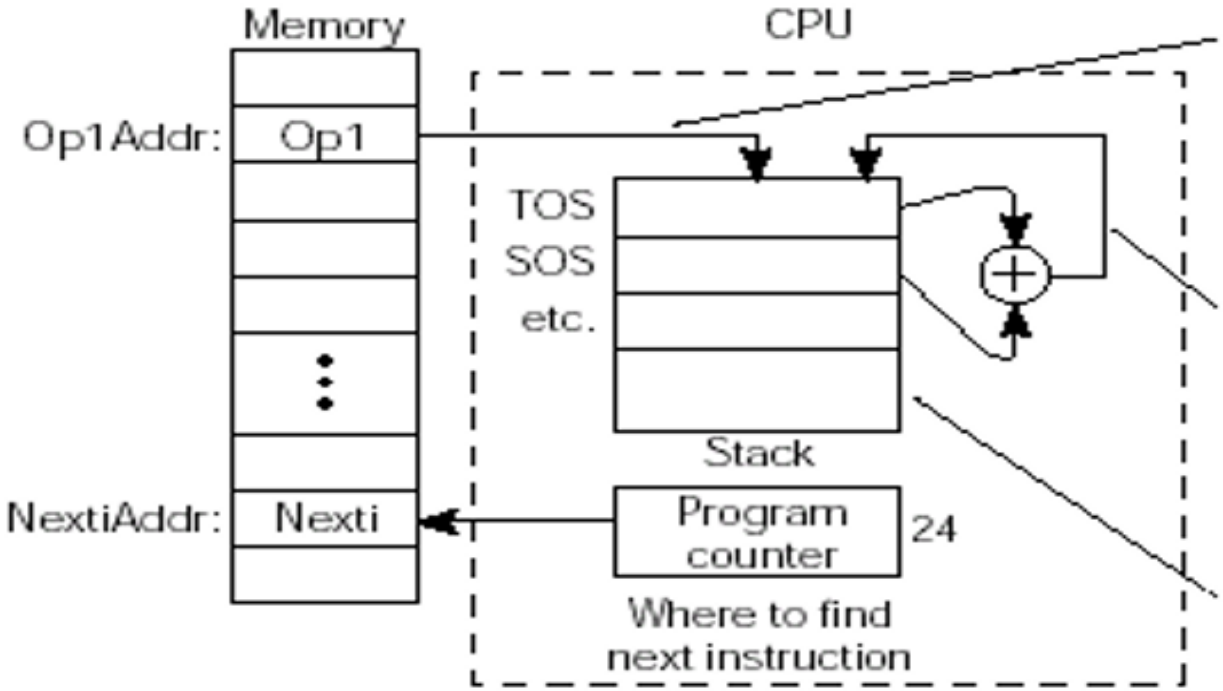
- Two words will be transferred to the CPU when the instruction itself is fetched = 2
- Then the one word representing the operand itself need to be fetched into the CPU register and the accumulator, supplies 1 operand and stores result = 1
- **Total=03**

0-Address Instruction

Fetch Instruction – 2 cycles
Fetch Operand – 1 cycle
(Uses stack to store when PUSH & hence no store in memory

- Uses a push down stack in CPU
- Arithmetic uses stack for both operands and the result

Computer must have a 1-address instruction to push and pop operands to and from the stack



Push Op1(TOS ← Op1)

Push	Op1Addr

Add (TOS ← TOS + SOS)

add

Where to find operands and where to put the result (on the Stack)

Fetch Instruction – 1 cycles, Fetch Operand & Execute – 0 cycle
(Uses stack to perform ADD & store the result in stack when ADD & No store in memory

Comparisons

Instruction Type	Memory To Store in Bytes	Memory To Encode in Bytes	M/As to fetch an Instruction	M/As to Execute an Instruction	Memory Traffic
4-address	5 x 3 = 15	1+(4 x 3) = 13	5	3	5+3=8
3-Address	4 x 3 = 12	1+(3 x 3) = 10	4	3	4+3=7
2-Address	3 x 3 = 09	1+(2 x 3) = 07	3	3	3+3=6
1-Address	2 x 3 = 06	1+(1 x 3) = 04	2	1	2+1=3
0-Address	1 x 3 = 03	1+(0 x 3) = 01	1 (ADD..) (or) 2 –(PUSH..POP)	0 (ADD..) (or) 1 –(PUSH..POP)	1+0=1

Problems – Example1

- Evaluate **a = (b+c)*d – e** in 3-, 2-, 1-, 0- address machines and compute the memory traffic. Assume 24 bit memory address and one byte opcode.

3-address	2-address	1-address	0-address

add a,b,c
mul a,a,d
sub a,a,e
	load a,b
	Add a,c
	Mul a,d
	Sub a,e
		Load b
		Add c
		Mul d
		Sub e
		Store a
			Push b
			Push c
			Add
			Push d
			Mul
			Push e
			Sub
			Pop a

Memory traffic for 3-address Machine: 7 * 3 = 21(Maximum)

Memory traffic for 2-address Machine: 6 * 4 = 24(Maximum)

Memory traffic for 1-address Machine: 3 * 5 = 15(Maximum)

Memory traffic for 0-address Machine: 3 * 5 + 3 = 18 (Maximum)

Three-address

add a, b, c	a←b+c
mpy a, a, d	a←a*d
sub a, a, e	a←a-e

Memory to Store	Memory to encode	M/As to Fetch	M/As to Execute	Memory Traffic
4*3=12	1+(3*3)=10	4	3	4+3=7
4*3=12	1+(3*3)=10	4	3	4+3=7
4*3=12	1+(3*3)=10	4	3	4+3=7
36	**30**	**12**	**9**	**21**

Two-address

load a, b	a←b
add a, c	a←a+c
mpy a, d	a←a*d
sub a, e	a←a-e

Memory to Store	Memory to encode	M/As to Fetch	M/As to Execute	Memory Traffic
3*3=9	1+(2*3)=7	3	2	3+2=5
3*3=9	1+(2*3)=7	3	3	3+3=6
3*3=9	1+(2*3)=7	3	3	3+3=6
3*3=9	1+(2*3)=7	3	3	3+3=6
36	**28**	**12**	**11**	**23**

One-address

load b	Acc←b
add c	Acc←Acc+c
mpy d	Acc←Acc*d
sub e	Acc←Acc-e
store a	a←Acc

Memory to Store	Memory to encode	M/As to Fetch	M/As to Execute	Memory Traffic
2*3=6	1+(1*3)=4	2	1	2+1=3
2*3=6	1+(1*3)=4	2	1	2+1=3
2*3=6	1+(1*3)=4	2	1	2+1=3
2*3=6	1+(1*3)=4	2	1	2+1=3
2*3=6	1+(1*3)=4	2	1	2+1=3
30	**20**	**10**	**5**	**15**

Zero-address

push b
push c
add
push d
mpy
push e
sub
pop a

Memory to Store	Memory to encode	M/As to Fetch	M/As to Execute	Memory Traffic
2*3=6	**1+(1*3)=4**	**2**	**1**	**3**
2*3=6	1+(1*3)=4	2	1	3
1*3=3	1	1	0	1
2*3=6	1+(1*3)=4	2	1	3
1*3=3	1	1	0	1
2*3=6	1+(1*3)=4	2	1	3
1*3=3	1	1	0	1
2*3=6	1+(1*3)=4	2	1	3
39	**23**	**13**	**5**	**18**

Problems – Example 2

Assume 24 bit memory address and one byte opcode.

- Evaluate `a = (b+c)*d-e`
- for 3- 2- 1- and 0-address machines

- What is size of program and amount of memory traffic in bytes?

Instructions

3-Address	2-Address	1-Address	0-Address
`add a, b, c`	`load a, b`	`lda b`	`push b`
`mult a, a, d`	`add a, c`	`add c`	`push c`
`sub a, a, e`	`mult a, d`	`mult d`	`add`
	`sub a, e`	`sub e`	`push d`
		`sta a`	`mult`
			`push e`
			`sub`
			`pop a`

Bytes size

	3-Address	2-Address	1-Address	0-Address
Instruction	30	28	20	23
Memory	27	33	15	15
Total	57	61	35	38

Practice problems

- Evaluate the expression a = b - c * d and compute memory traffic for 4, 3, 2, 1,and 0 address machine. Assuming that addresses are 16 bits, data values are 16 bits, opcodes are 8 bits and 1 byte word length.
- Develop a comparative table for the performance parameters such as memory to store, memory to encode, M/As to fetch , M/As to execute and total memory Traffic for 4-,3-,2-,1-,0- address machine instructions. Consider the following specifications: Memory word size is 1 byte, Memory/register Address size is 2byte, Opcode size is 1 byte.

i. Write an appropriate assembly language programming using 3-Address, 2-Address, 1-Address and 0-address machine instructions for the following expression (with registers & without registers). Assume that all are integer operations.

$$X= (A / B + C * D) / (D * E - F + C / A) + G$$

ii. Compute various performance factors such as memory to store a program, memory to encode a whole program, Memory access to fetch & execute and memory traffic.

Addressing Modes

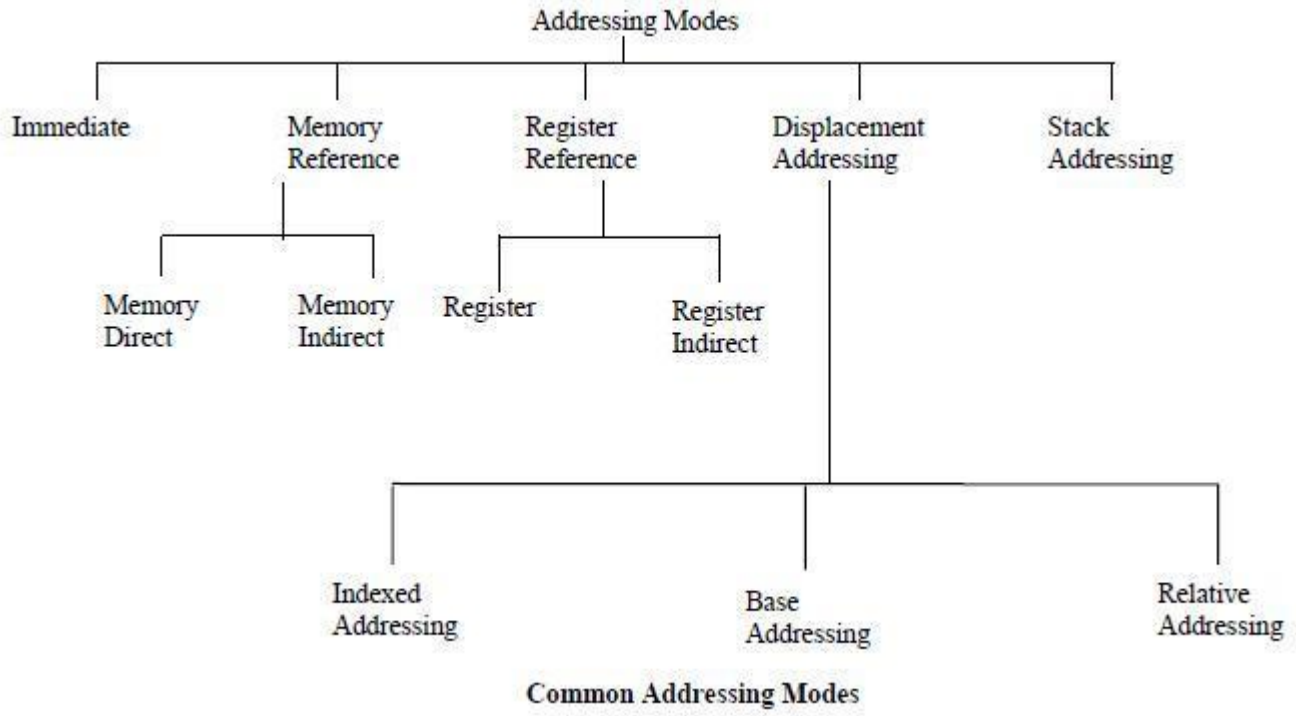
- Instruction Set is categorized **Based on Addressing Modes**
- **Addressing Mode**
  - **The different ways in which the location of an operand is specified in an instruction are referred to as addressing modes.**
  - The mode of access of effective address is called addressing mode
  - The Way the operands are specified in the instruction
  - The operation to be performed is indicated by the opcode.
  - Operands can be in registers, memory or embedded in the instruction
- **Effective Address**
  - The address in which the actual operand is available is called as Effective address

Addressing Modes

- **Terminologies**
  - **Displacement:** It is an 8 bit or 16 bit immediate value given in the instruction.
  - **Base**: Contents of base register, BX or BP.
  - **Index**: Content of index register SI or DI.

Classification of Addressing Modes

1. Stack (Implied/Implicit) Addressing mode
2. Immediate Addressing mode
3. (Memory) Direct Addressing mode
4. Register Direct Addressing mode
5. (Memory) Indirect Addressing mo
6. Register Indirect Addressing mode
7. Displacement Addressing modes
    1. Indexed Addressing mode
    2. Base register Addressing mode
    3. Relative Addressing Mode
9. Auto Increment Addressing Mode
10. Auto Decrement Addressing Mode



Common Addressing Modes

1.Stack (Implied/Implicit) Addressing mode

- Definition of the instruction itself specify the operands implicitly.
- Operand is implied / specified implicitly in the instruction itself.
- Operations like PUSH and POP for the computation
- Zero address instructions in a stack organized computer are implied mode instructions.
- Effective Address **(EA) = AC or Stack[SP]**
- Advantage:
  - Instruction specifies a fixed and unvarying address
  - No memory references
- Disadvantage:
  - Limited Computational Capacity

DEC (Decrement A register)

CLC (used to reset Carry flag to 0)

PUSH

POP

2.Immediate Addressing mode

- The simplest form of addressing
- Effective Address **(EA) = Value**
- Data is a part of instruction itself.
- **Example:**
  - **MOVE #100, R1**
    - Here the data 100 is moved to R1.
  - **MVI #01, A**
    - MVI stands for Move Immediate. This basically implies move 01 to A.
- **Advantage:**
  - This mode can be used to define and use constants or set of initial values of variables.
  - No memory references
- **Disadvantage:**
  - Limited Operand size

(a) Immediate addressing:

instruction contains
the operand

3. (Memory) Direct Addressing mode

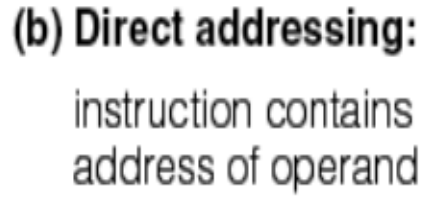
- The address where data is available is part of the instruction
- The address field contains the effective address of the operand
- Effective Address **(EA) = LOC**
- **Example:**
  - **MOVE A, R1**
    - Here the data in memory location A is moved to R1.

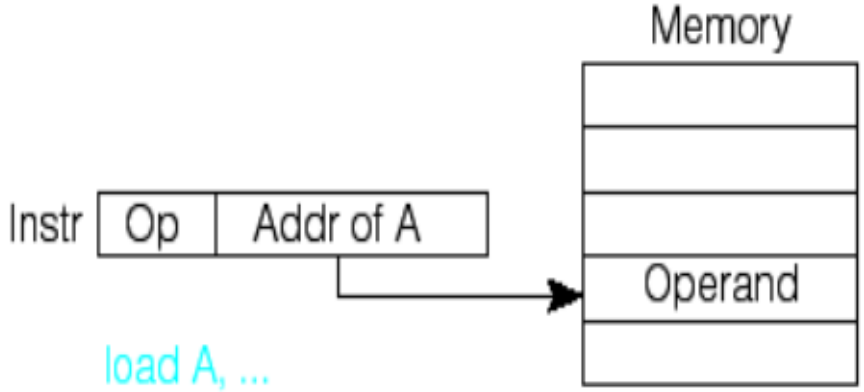
Advantage:

- Large operand Magnitude, Simple

Disadvantage:

- Limited Address Size
- The change in the location of the program is associated with the change in all absolute memory references.





4. Register Direct Addressing mode

- Register addressing is similar to direct addressing
  - The only difference is that the address field refers to a register rather than a main memory address
- Effective Address **(EA) = Ri**
- **Example:**
  - **MOVE R2, R1**
    - Here the data in Register R2 is moved to R1.

Advantage:

- No memory Reference

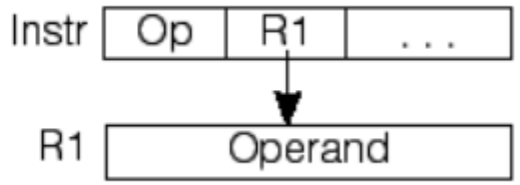
Disadvantage:

- Limited number of registers

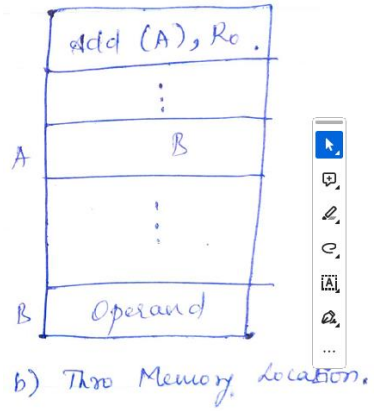
(d) Register direct addressing:

register contains operand

load R1, ...



5. (Memory) Indirect Addressing mode



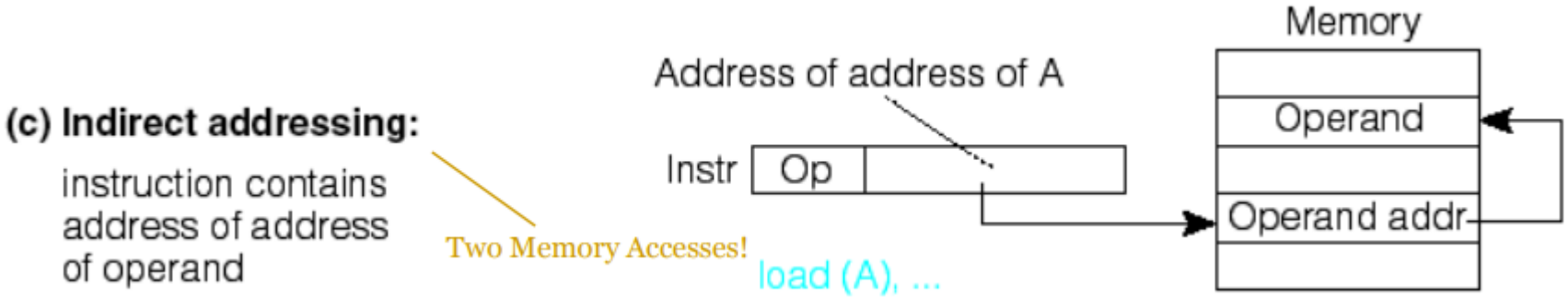
- The address field contains the address of effective address of the operand
  - Contains a full –length address of the operand
- Effective Address **(EA) = (LOC) or [LOC]**
- **Example:**
  - **MOVE (A), R1**
    - Here A – has another memory address (not data)
    - The data in the address in A is moved to R1.

Advantage:

- Large address space

Disadvantage:

- The change in the location of the program is associated with the change in all absolute memory references



6. Register Indirect Addressing mode

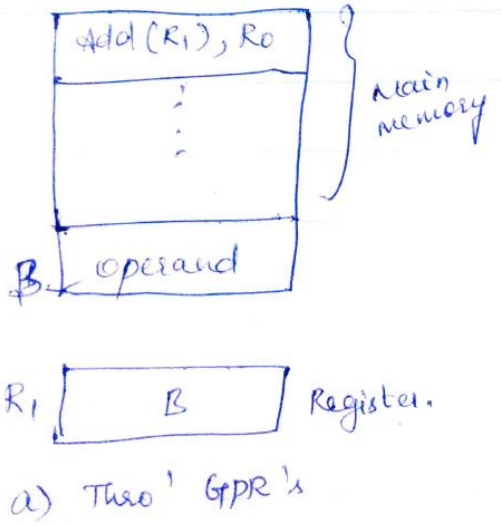
- Register indirect is just analogous to indirect addressing in both cases
  - The only difference is whether the address field refers to memory location or a register.
- Effective Address **(EA) = (Ri) or [Ri]**
- **Example:**
  - **MOVE (R2), R1**
    - Here R2 – has memory address (not data)
    - The data in the address in R2 is moved to R1.

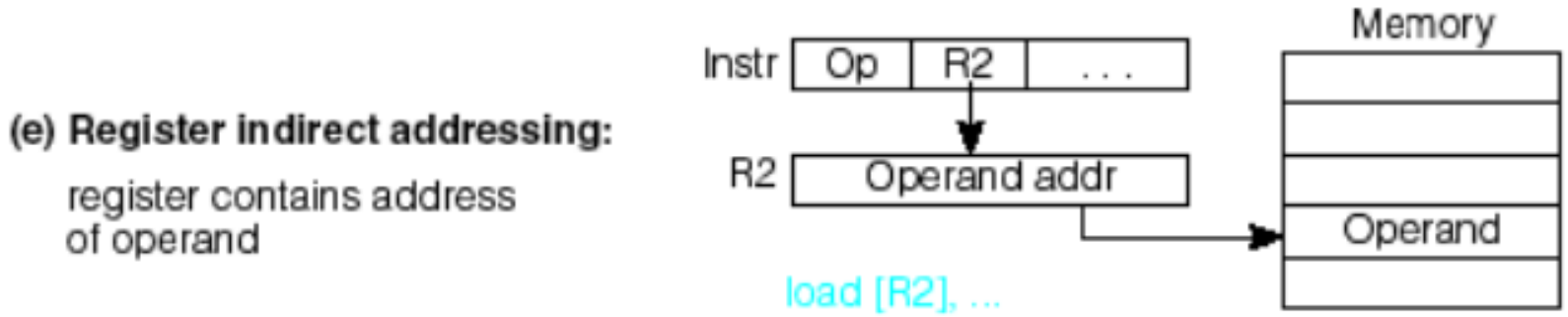
Advantage:

- Large address space

Disadvantage:

- Extra memory space





7. Displacement Addressing modes - Indexed Addressing mode

- The address field reference a main memory address, and the referenced register contain a positive displacement from that address .
- Effective Address **(EA) = (Ri) + X**

Index Register

Index value of an array Stored in

Index Register

The base register holds the beginning location of a memory array, while the index register holds the relative position of an element in the array.

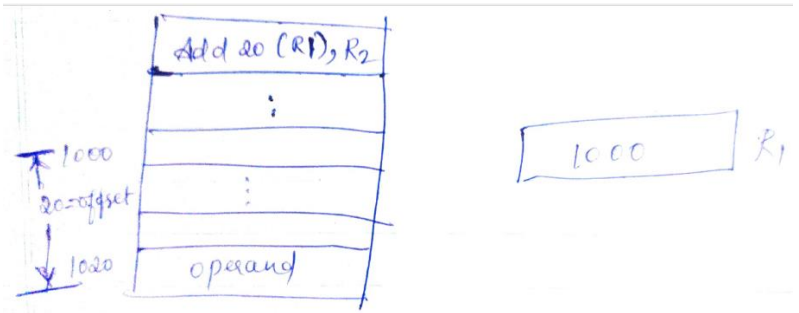
Advantage:

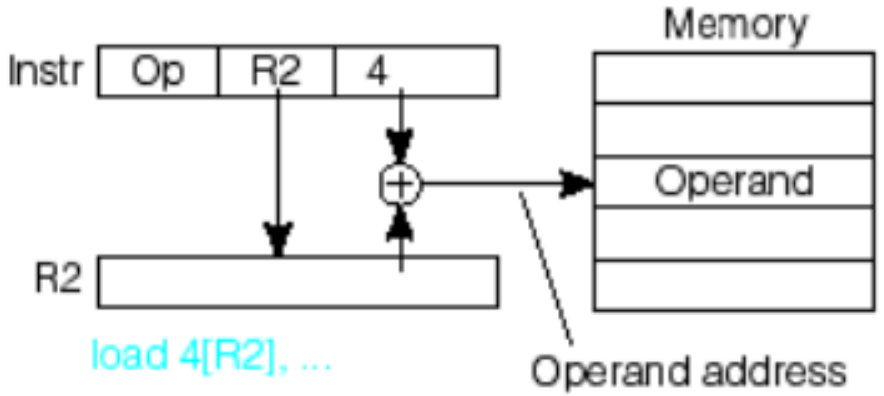
- Special Locality

- **Example:**
  - **MOVE 20 (R2), R1**
    - Here R2 – has memory address (not data).
    - The address in R2 is added with the index value 20 which is the EA.
    - The data in the address in R2+20 is moved to R1.

(f) Displacement (based or indexed) addressing:

address of operand = register + constant





Example: Works for ARRAYS

7. Displacement Addressing modes – Base register Addressing mode

- The address field reference a main memory address, and the referenced register contain a positive displacement from that address .
- Effective Address **(EA) = (Ri + BX)**
- **Example:**
  - **ADD AX, [BX+SI]**
  - **ADD R1, (R2+R3)**
    - **Meaning: R1 ⬅ R1+M[R2+R3]**
    - Here R2 & BX – Base Register
    - R3 & SI – Index register
- EA=sum (content of BR and SI)

ADD R1, (R2+3) --- Base
ADD R1,(R2+R3) --- Base with Index
ADD R1, 20(R2+R3) --- Base with Index and Offset

Base Register

The base register holds the beginning location of a memory array, while the index register holds the relative position of an element in the array.

Advantage:

It use a convenient means of implementing segmentation.

Disadvantage:

Complexity

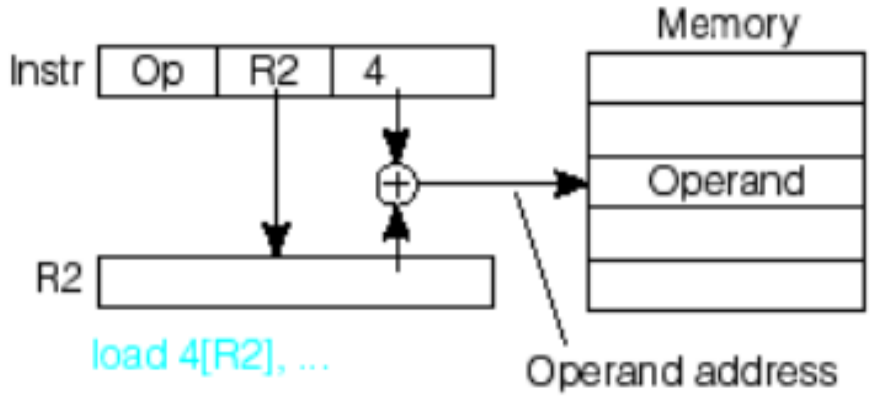
Base address of the array Stored

Base Register

Example: Works for ARRAYS

(f) Displacement (based or indexed) addressing:

address of operand = register + constant



7. Displacement Addressing modes – Relative Addressing Mode

- **PC**- relative addressing - the implicitly referenced register is the program counter (PC)
- The effective address is the offset parameter added to the address of the next instruction.
- Effective Address **(EA) = (PC) + X**
- **Example:**
  - **If Branch > 0,**

    JUMP -200

    - Here, based on the condition,

    the **address in PC is incremented with constant value 200.**

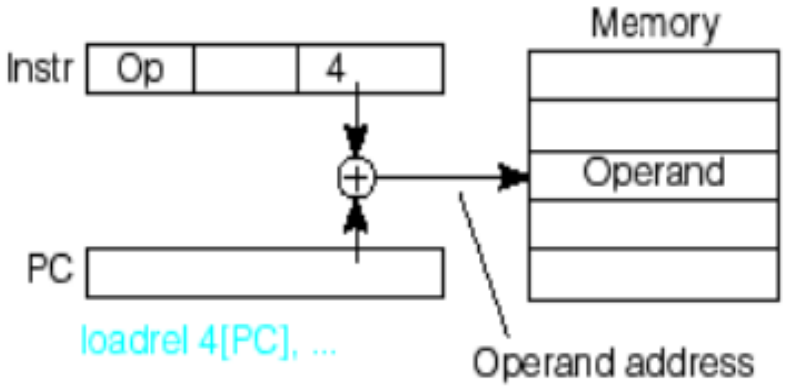
Advantage:

- program-relative addressing is that the code may be position-independent

Disadvantage:

- Complexity

(g) Relative addressing:
address of operand = PC + constant



8. Auto Increment Addressing mode

- Register incremented after accessing memory
- The effective address is the offset parameter added to the address of the next instruction.
- Effective Address **(EA) = (Ri); Increment**
- **Example:**
  - **ADD (R2)+, R0**
    - Here R2 – has Operand Address
    - After accessing the operand, the register content is automatically incremented.

Advantage:

- Useful while transferring large chunks of contiguous data.

Disadvantage:

- Complexity

9. Auto Decrement Addressing mode

- Register decremented and then contents accessed memory
- The effective address is the offset parameter added to the address of the next instruction.
- Effective Address **(EA) = Decrement; (Ri)**
- **Example:**
  - **ADD -(R2), R0**
    - Here R2 after decrement – has Operand Address
    - The contents of Register is decremented first and then the content gives Operand Address.

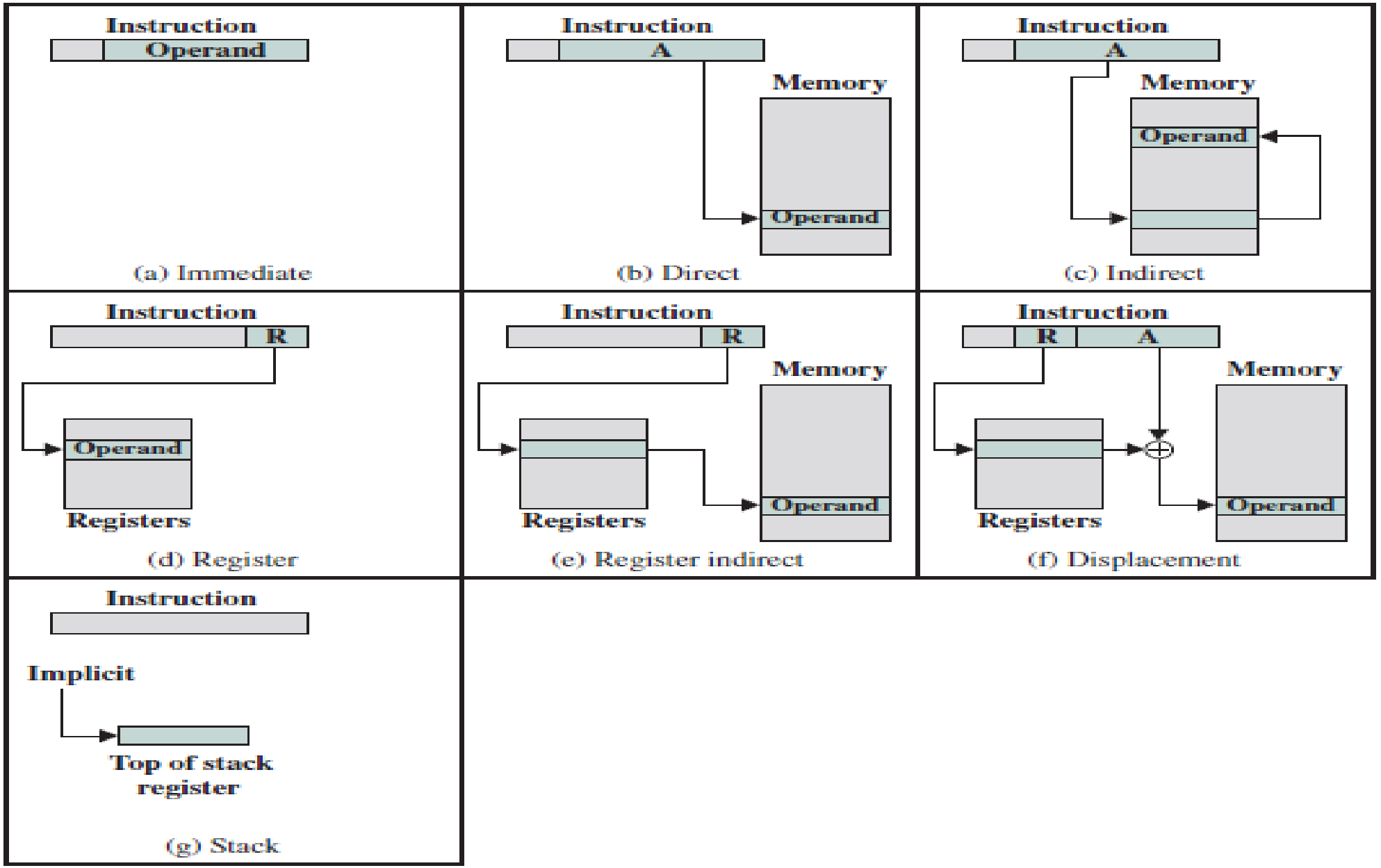
Advantage:

- Useful while transferring large chunks of contiguous data.

Disadvantage:

- Complexity

Addressing modes



Addressing modes

	Mode	Algorithm	Advantage	Disadvantage
1	Implied	Operand=Φ	No Memory references	Limited Computation capability
2	Immediate	Operand=1	No memory Reference	Limited operand magnitude
3	Direct	EA = A	Simple	Limited address space
4	Indirect	EA =(A)	Large Address space	Multiple Memory references
5	Register	EA = R	No memory Reference	Limited address space
6.1	Register Indirect	EA = (R)	Large address space	Extra memory space
6.2	Relative	EA = PC+2's Complement	Use for control instructions	Complexity
6.3	Base Register	EA=BA+ 2's Complement	convenient during segmentation	Complexity
6.4	Indexed	EA= A+ Index value	Accessing array elements	Array lower bounds
7	Stack	EA= Top of Stack	No memory Reference	Limited Applicability

Problems

Find the effective address and the content of AC for the given data.

PC = 200
R1 = 400
XR = 100
AC

Address	Memory	
200	Load to AC	Mode
201	Address = 500	
202	Next instruction	
399	450	
400	700	
500	800	
600	900	
702	325	
800	300	

Addressing Mode	Effective Address		Content of AC
Direct Address	**500**	**AC ← (500)**	**800**
Immediate operand	**201**	**AC ← 500**	**500**
Indirect address	**800**	**AC ← ((500))**	**300**
Relative address	**702**	**AC ← (PC + 500)**	**325**
Indexed address	**600**	**AC ← (XR + 500)**	**900**
Register	**-**	**AC ← R1**	**400**
Register Indirect	**400**	**AC ← (R1)**	**700**
Autoincrement	**400**	**AC ← (R1)+**	**700**
Autodecrement	**399**	**AC ← -(R1)**	**450**

Questions

- An instruction is stored at location 300 with its address field at location 301. The address field has the value 400. A processor register R1 contains the number 200. Evaluate the effective address if the addressing mode of the instruction is (a) direct; (b) immediate (c) relative (d) register indirect; (e) index with R1 as the index register.
- Let the address stored in the program counter be designated by the symbol X1. The instruction stored in X1 has the address part (operand reference) X2. The operand needed to execute the instruction is stored in the memory word with address X3. An index register contains the value X4. What is the relationship between these various quantities if the addressing mode of the instruction is
- (a) direct (b) indirect (c) PC relative (d) indexed?